



CIÊNCIA DA
COMPUTAÇÃO
GRANDE FLORIANÓPOLIS



Curso de Ciência da Computação

Programação Orientada a Objetos – POO

Profa. Fernanda dos Santos Cunha

Programas exemplo retirados de **Java: como programar**
Deitel, P.; Deitel, H. - São Paulo: Pearson Education do Brasil, 2017.
Conteúdo baseado nos cap 8 e 10.

Orientação a Objetos

Atributo de objeto



- **Atributo de objeto** (variável de instância) é uma característica relevante para o objeto.
 - Dica: ele deve ser assim declarado somente quando a necessidade de armazenar esta informação for maior que a duração de uma operação.
 - Logo, existem “atributos calculados” (p.ex. os vencimentos calculados pelo método `earnings`).
 - Cada objeto terá sua própria cópia (com respectivo valor) do atributo.

Atributo de objeto *final*

- O **Princípio do Menor Privilégio** é fundamental para uma boa Engenharia de Software:

“Deve ser concedido ao código somente a quantidade de privilégio e acesso que ele precisa para realizar sua tarefa designada, mas não mais que isso.”

=> Programas mais robustos, evitando que o código modifique acidentalmente (ou maliciosamente) os valores das variáveis e chame métodos que *não* deveriam estar acessíveis.

Atributo de objeto *final*

- Algumas variáveis de instância precisam ser *modificáveis* e **algumas não**.
 - Usar a palavra-chave `final` para especificar o fato de que a variável de instancia ***não será modificável*** (será *constante*) e qualquer tentativa de modificá-la será um erro.
- As variáveis `final` podem ser inicializadas quando declaradas, e se não forem, *devem* ser inicializadas em cada construtor da classe, permitindo que cada objeto da classe tenha um valor diferente para a constante.
 - Se a variável `final` *não* é inicializada na sua declaração ou em cada construtor, ocorre um erro de compilação.

Métodos e classes `final`

- Os métodos `final` não podem ser sobrescritos.
 - Métodos `private` e métodos `static` são implicitamente `final`.
 - As subclasses utilizam a mesma implementação
 - Chamadas a métodos `final` são resolvidas em tempo de compilação (**ligação estática**, veremos adiante).
- Classes `final` não podem ser superclasses.
 - Uma classe `final` não pode ser estendida para criar uma subclasse e todos os seus métodos são implicitamente `final`.
- Para obter mais informações sobre a utilização de `final`:
<http://docs.oracle.com/javase/tutorial/java/IandI/final.html>

Orientação a Objetos

Atributo de classe



- Existem informações (variáveis) para as quais cada objeto de uma classe *não* precisa de sua própria cópia separada.
 - Conhecido por **atributo de classe ou atributo estático** (declara-se `static`).
 - Representa informações de escopo de classe (p.ex. um contador de objetos da classe).
 - Todos os objetos compartilham e acessam o mesmo atributo/valor.
 - Na UML o atributo de classe fica sublinhado.
- Atributos estáticos podem ser acessados somente por **métodos da classe ou estáticos**.

Orientação a Objetos

Operação de classe



- Uma **operação de classe** é um serviço que pode ser invocado diretamente a partir da classe, realizando uma tarefa que não depende de um objeto específico.
 - Conhecido por **método de classe ou método estático** (declara-se `static`).
 - Pode ser invocado quando não há nenhum objeto da classe, basta fazer `NomeClasse.metodoEstatico()`.
 - Na UML o método de classe fica sublinhado.

Orientação a Objetos

Membros `static`



- Deve-se ter na classe um método estático para poder acessar um atributo estático quando ainda não existirem objetos da classe.
- Restrição: um método estático não pode acessar as variáveis de instância e os métodos de instância de uma classe, porque ele pode ser chamado mesmo quando nenhum objeto da classe foi instanciado.
 - Logo, a referência `this` **não pode** ser utilizada em um método estático, pois ela se refere a um objeto específico.


```
1 // Figura 8.12: Employee.java
2 // Variável static utilizada para manter uma contagem do número de
3 // objetos Employee na memória.
4
5 public class Employee
6 {
7     private static int count = 0; // número de Employees criados
8     private String firstName;
9     private String lastName;
10
11     // inicializa Employee, adiciona 1 a static count e
12     // gera a saída de String indicando que o construtor foi chamado
13     public Employee(String firstName, String lastName)
14     {
15         this.firstName = firstName;
16         this.lastName = lastName;
17
18         ++count; // incrementa contagem estática de empregados
19         System.out.printf("Employee constructor: %s %s; count = %d\n",
20             firstName, lastName, count);
21     }
22
23     // obtém o primeiro nome
24     public String getFirstName()
25     {
26         return firstName;
27     }
28
29     // obtém o último nome
30     public String getLastName()
31     {
32         return lastName;
33     }
34
35     // método estático para obter valor de contagem de estática
36     public static int getCount()
37     {
38         return count;
39     }
40 } // fim da classe Employee
```

Atributo estático

Java - Membros static

Método estático

Java - Membros static

```
1 // Figura 8.13: EmployeeTest.java
2 // Demonstração do membro static.
3
4 public class EmployeeTest
5 {
6     public static void main(String[] args)
7     {
8         // mostra que a contagem é 0 antes de criar Employees
9         System.out.printf("Employees before instantiation: %d\n",
10             Employee.getCount());
11
12         // cria dois Employees; a contagem deve ser 2
13         Employee e1 = new Employee("Susan", "Baker");
14         Employee e2 = new Employee("Bob", "Blue");
15
16         // mostra que a contagem é 2 depois de criar dois Employees
17         System.out.printf("\nEmployees after instantiation:\n");
18         System.out.printf("via e1.getCount(): %d\n", e1.getCount());
19         System.out.printf("via e2.getCount(): %d\n", e2.getCount());
20         System.out.printf("via Employee.getCount(): %d\n",
21             Employee.getCount());
22
23         // obtém nomes de Employees
24         System.out.printf("\nEmployee 1: %s %s\nEmployee 2: %s %s\n",
25             e1.getFirstName(), e1.getLastName(),
26             e2.getFirstName(), e2.getLastName());
27     }
28 } // fim da classe EmployeeTest
```

Employees before instantiation: 0
Employee constructor: Susan Baker; count = 1
Employee constructor: Bob Blue; count = 2
Employees after instantiation:
via e1.getCount(): 2
via e2.getCount(): 2
via Employee.getCount(): 2

Employee 1: Susan Baker
Employee 2: Bob Blue

Orientação a Objetos

Classe Abstrata

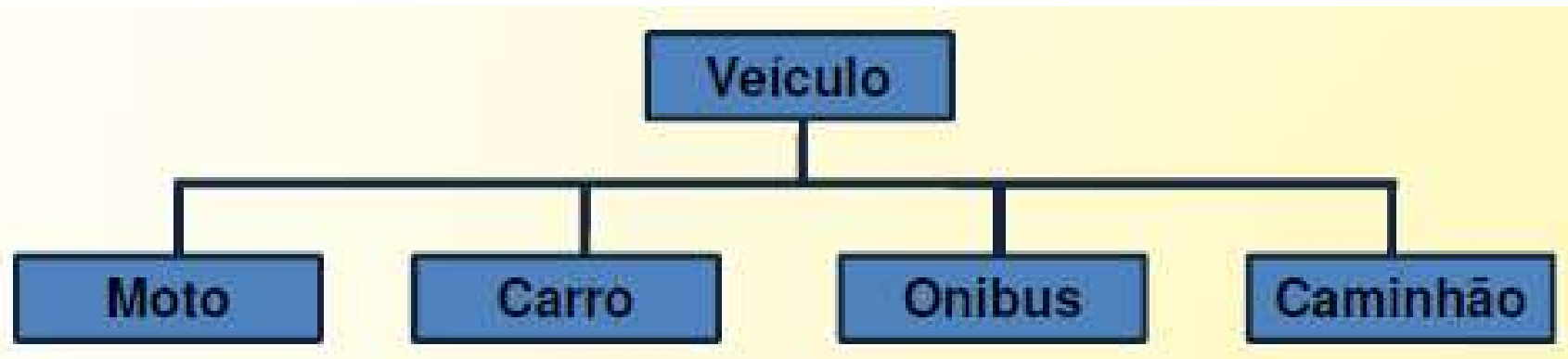


- Toda classe que pode instanciar objetos a partir dela é chamada **classe concreta**.
- Porém, pode haver uma classe que nunca vai gerar objetos dentro do sistema => chamada **classe abstrata**.
 - Ex.: num sistema acadêmico, faz sentido criar objetos da classe Pessoa?
 - Se não faz, então Pessoa é uma classe abstrata.
- Assim, a classe abstrata é uma abstração parcial de algo do sistema e por isso não faz sentido instanciar objetos a partir dela.
- Na UML a classe abstrata tem seu nome em itálico.

Orientação a Objetos

Classe Abstrata

- Motivação
 - Base para uma hierarquia de herança
 - Manipulação de objetos utilizando recurso de polimorfismo (veremos em breve)
 - Forma para definir classes genéricas representativas de um superconjunto de subclasses, que irão representar os objetos em si



Orientação a Objetos

Classe Abstrata



- As classes abstratas são *incompletas*. As subclasses devem declarar as “partes ausentes” para que se tornem classes “concretas”, a partir das quais pode-se instanciar objetos. Do contrário, essas subclasses também serão abstratas.
- O propósito de uma classe abstrata é fornecer uma superclasse apropriada a partir da qual outras classes podem herdar e assim compartilhar um *design* comum.
- Nem todas as hierarquias contêm classes abstratas. Mas, as classes abstratas às vezes constituem vários níveis da hierarquia.

Java

Classe Abstrata

- Em Java, para criar uma Classe Abstrata basta usar a palavra reservada **abstract** na sua declaração

```
public abstract class NomeClasse {  
    ...  
}
```

- Uma classe abstrata normalmente contém um ou mais métodos abstratos (que **não tem** implementação).

```
public abstract void draw();
```

- Cada subclasse concreta de uma superclasse abstrata também deve fornecer implementações concretas de cada um dos métodos abstratos da superclasse.

Java

Classe Abstrata



ATENÇÃO:

- Os construtores e métodos estáticos não podem ser declarados `abstract`.
 - Os construtores *não* são herdados, portanto um construtor `abstract` nunca seria implementado.
 - Métodos `abstract` devem ser sobrescritos para que possam processar objetos com base em seus tipos - não faria sentido declarar um método `static` como `abstract`.

Java

Classe Abstrata

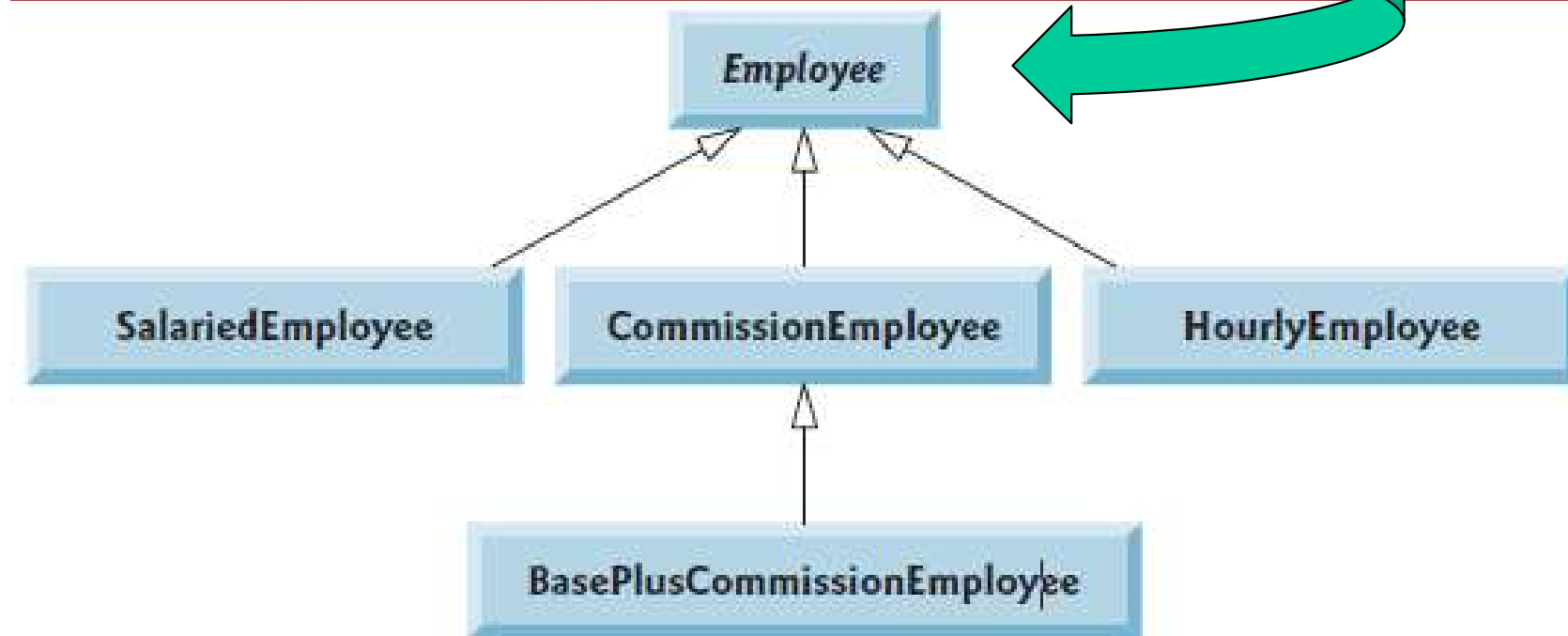
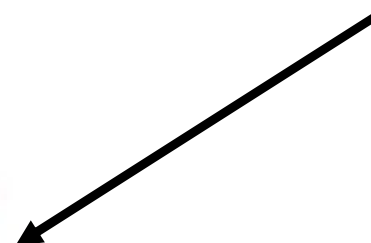


Figura 10.2 | Diagrama de classes da UML da hierarquia Employee.

Java Classe Abstrata

```
1 // Figura 10.4: Employee.java
2 // Superclasse abstrata Employee.
3
4 public abstract class Employee
5 {
6     private final String firstName;
7     private final String lastName;
8     private final String socialSecurityNumber;
9
10    // construtor
11    public Employee(String firstName, String lastName,
12        String socialSecurityNumber)
13    {
14        this.firstName = firstName;
15        this.lastName = lastName;
16        this.socialSecurityNumber = socialSecurityNumber;
17    }
18
19    // retorna o nome
20    public String getFirstName()
21    {
22        return firstName;
23    }
24
25    // retorna o sobrenome
26    public String getLastName()
27    {
28        return lastName;
29    }
30
31    // retorna o número do seguro social
32    public String getSocialSecurityNumber()
33    {
34        return socialSecurityNumber;
35    }
36
37    // retorna a representação de String do objeto Employee
38    @Override
39    public String toString()
40    {
41        return String.format("%s %s\nsocial security number: %s",
42            getFirstName(), getLastName(), getSocialSecurityNumber());
43    }
44
45    // O método abstract deve ser sobrescrito pelas subclasses concretas
46    public abstract double earnings(); // nenhuma implementação aqui
47 }
```

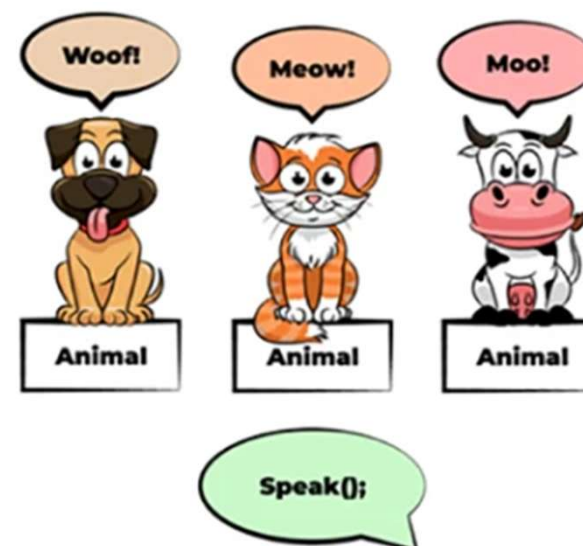
Método abstrato



Orientação a Objetos

Polimorfismo

- Do grego *poli* + *morphos* = **muitas formas**
- 4º pilar da POO, após abstração, encapsulamento e herança.
- Em particular, **permite escrever programas que processam objetos** que compartilham a mesma superclasse, direta ou indiretamente, **como se todos fossem objetos da superclasse** – o que pode simplificar a programação.



Orientação a Objetos

Polimorfismo



- Permite “programar no geral” em vez de “programar no específico” reduzindo a necessidade de condicionais no programa.
- Oferece uniformidade na interface.
- Implementação de sistemas facilmente extensíveis.
 - Novas classes são incluídas com pouca/nenhuma modificação a partes gerais do programa, desde que elas façam parte da hierarquia.
 - Uma subclasse também pode adicionar atributos, operações e relacionamentos; redefinir operações herdadas (`overriding`: nova implementação mantendo a mesma assinatura da operação original).

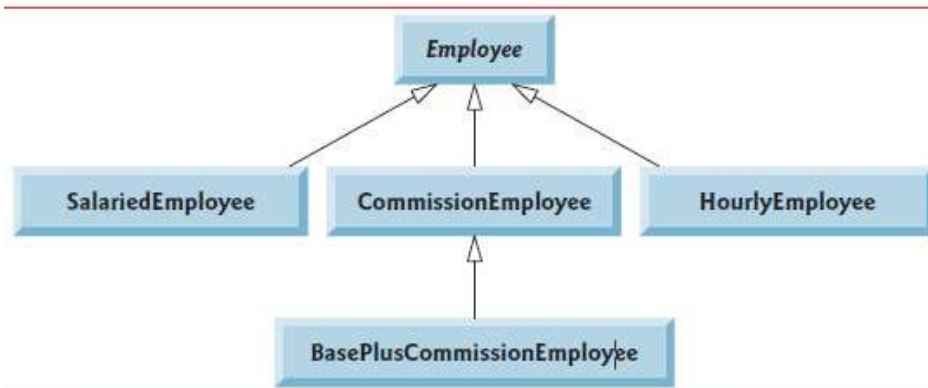


Figura 10.2 | Diagrama de classes da UML da hierarquia Employee.

Java

Polimorfismo

Usar um método abstrato e o polimorfismo para realizar cálculos da folha de pagamento, atendendo aos seguintes requisitos:

Uma empresa paga semanalmente seus 4 tipos de funcionários:

- Assalariados: salários fixos semanais
- Horistas: trabalham por hora e recebem horas extras (1,5 vez sua taxa de salário/hora) pelas horas trabalhadas além das 40h normais
- Comissionados: recebem porcentagem sobre suas vendas
- Assalariados-comissionados: recebem salário-base mais porcentagem sobre suas vendas.

A empresa decidiu recompensar os Assalariados-comissionados adicionando 10% aos seus salários-base.

Escreva um aplicativo que realiza os cálculos da folha de pagamento polimorficamente.

Java

Polimorfismo

	earnings	toString
Employee	abstract	<i>firstName lastName</i> social security number: <i>SSN</i>
Salaried-Employee	weeklySalary	salaried employee: <i>firstName lastName</i> social security number: <i>SSN</i> weekly salary: <i>weeklySalary</i>
Hourly-Employee	<pre>if (hours <= 40) wage * hours else if (hours > 40) { 40 * wage + (hours - 40) * wage * 1.5 }</pre>	hourly employee: <i>firstName lastName</i> social security number: <i>SSN</i> hourly wage: <i>wage</i> ; hours worked: <i>hours</i>
Commission-Employee	commissionRate * grossSales	commission employee: <i>firstName lastName</i> social security number: <i>SSN</i> gross sales: <i>grossSales</i> ; commission rate: <i>commissionRate</i>
BasePlus-Commission-Employee	(commissionRate * grossSales) + baseSalary	base salaried commission employee: <i>firstName lastName</i> social security number: <i>SSN</i> gross sales: <i>grossSales</i> ; commission rate: <i>commissionRate</i> ; base salary: <i>baseSalary</i>

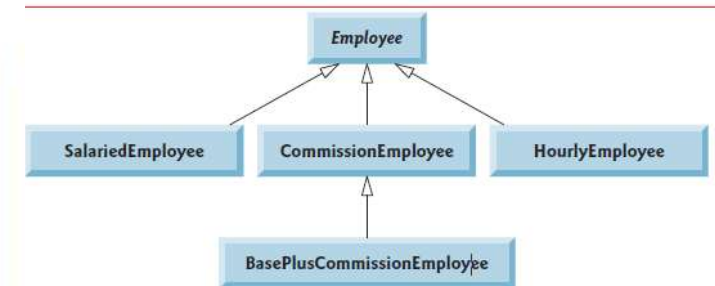


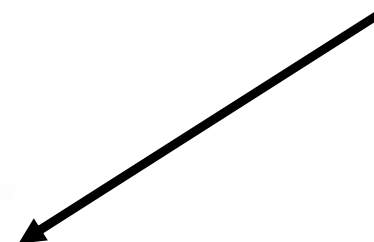
Figura 10.2 | Diagrama de classes da UML da hierarquia Employee.

Figura 10.3 | Interface polimórfica para as classes na hierarquia Employee.

```
1 // Figura 10.4: Employee.java
2 // Superclasse abstrata Employee.
3
4 public abstract class Employee
5 {
6     private final String firstName;
7     private final String lastName;
8     private final String socialSecurityNumber;
9
10    // construtor
11    public Employee(String firstName, String lastName,
12        String socialSecurityNumber)
13    {
14        this.firstName = firstName;
15        this.lastName = lastName;
16        this.socialSecurityNumber = socialSecurityNumber;
17    }
18
19    // retorna o nome
20    public String getFirstName()
21    {
22        return firstName;
23    }
24
25    // retorna o sobrenome
26    public String getLastName()
27    {
28        return lastName;
29    }
30
31    // retorna o número do seguro social
32    public String getSocialSecurityNumber()
33    {
34        return socialSecurityNumber;
35    }
36
37    // retorna a representação de String do objeto Employee
38    @Override
39    public String toString()
40    {
41        return String.format("%s %s\nsocial security number: %s",
42            getFirstName(), getLastName(), getSocialSecurityNumber());
43    }
44
45    // O método abstract deve ser sobrescrito pelas subclasses concretas
46    public abstract double earnings(); // nenhuma implementação aqui
47 } // fim da classe abstrata Employee
```

Declarar earnings
como um método
abstract pois não faz
sentido fornecer uma
implementação
específica desse método
nesta classe.

Método abstrato




```
1 // Figura 10.5: SalariedEmployee.java
2 // A classe concreta SalariedEmployee estende a classe abstrata Employee.
3
4 public class SalariedEmployee extends Employee
5 {
6     private double weeklySalary;
7
8     // construtor
9     public SalariedEmployee(String firstName, String lastName,
10         String socialSecurityNumber, double weeklySalary)
11     {
12         super(firstName, lastName, socialSecurityNumber);
13
14         if (weeklySalary < 0.0)
15             throw new IllegalArgumentException(
16                 "Weekly salary must be >= 0.0");
17
18         this.weeklySalary = weeklySalary;
19     }
20
21     // configura o salário
22     public void setWeeklySalary(double weeklySalary)
23     {
24         if (weeklySalary < 0.0)
25             throw new IllegalArgumentException(
26                 "Weekly salary must be >= 0.0");
27
28         this.weeklySalary = weeklySalary;
29     }
30
31     // retorna o salário
32     public double getWeeklySalary()
33     {
34         return weeklySalary;
35     }
36
37     // calcula os rendimentos: sobrescreve o método earnings em Employee
38     @Override
39     public double earnings()
40     {
41         return getWeeklySalary();
42     }
43
44     // retorna a representação String do objeto SalariedEmployee
45     @Override
46     public String toString()
47     {
48         return String.format("salaried employee: %s\n%s: $%,.2f",
49             super.toString(), "weekly salary", getWeeklySalary());
50     }
51 } // fim da classe SalariedEmployee
```

Método abstrato da
superclasse implementado

Método toString
redefinido

```

1 // Figura 10.6: HourlyEmployee.java
2 // Classe HourlyEmployee estende Employee.
3
4 public class HourlyEmployee extends Employee
5 {
6     private double wage; // salário por hora
7     private double hours; // horas trabalhadas durante a semana
8
9     // construtor
10    public HourlyEmployee(String firstName, String lastName,
11        String socialSecurityNumber, double wage, double hours)
12    {
13        super(firstName, lastName, socialSecurityNumber);
14
15        if (wage < 0.0) // valida remuneração
16            throw new IllegalArgumentException(
17                "Hourly wage must be >= 0.0");
18
19        if ((hours < 0.0) || (hours > 168.0)) // valida horas
20            throw new IllegalArgumentException(
21                "Hours worked must be >= 0.0 and <= 168.0");
22
23        this.wage = wage;
24        this.hours = hours;
25    }
26
27    // configura a remuneração
28    public void setWage(double wage)
29    {
30        if (wage < 0.0) // valida remuneração
31            throw new IllegalArgumentException(
32                "Hourly wage must be >= 0.0");
33
34        this.wage = wage;
35    }
36

```

```

37 // retorna a remuneração
38 public double getWage()
39 {
40     return wage;
41 }
42
43 // configura as horas trabalhadas
44 public void setHours(double hours)
45 {
46     if ((hours < 0.0) || (hours > 168.0)) // valida horas
47         throw new IllegalArgumentException(
48             "Hours worked must be >= 0.0 and <= 168.0");
49
50     this.hours = hours;
51 }
52
53 // retorna as horas trabalhadas
54 public double getHours()
55 {
56     return hours;
57 }
58
59 // calcula os rendimentos; sobrescreve o método earnings em Employee
60 @Override
61 public double earnings()
62 {
63     if (getHours() <= 40) // nenhuma hora extra
64         return getWage() * getHours();
65     else
66         return 40 * getWage() + (getHours() - 40) * getWage() * 1.5;
67 }
68
69 // retorna a representação de String do objeto HourlyEmployee
70 @Override
71 public String toString()
72 {
73     return String.format("hourly employee: %s\n%s: $%,.2f; %s: $%,.2f",
74         super.toString(), "hourly wage", getWage(),
75         "hours worked", getHours());
76 }
77 } // fim da classe HourlyEmployee

```



```

1 // Figura 10.7: CommissionEmployee.java
2 // Classe CommissionEmployee estende Employee.
3
4 public class CommissionEmployee extends Employee
5 {
6     private double grossSales; // vendas brutas semanais
7     private double commissionRate; // porcentagem da comissão
8
9     // construtor
10    public CommissionEmployee(String firstName, String lastName,
11        String socialSecurityNumber, double grossSales,
12        double commissionRate)
13    {
14        super(firstName, lastName, socialSecurityNumber);
15
16        if (commissionRate <= 0.0 || commissionRate >= 1.0) // valida
17            throw new IllegalArgumentException(
18                "Commission rate must be > 0.0 and < 1.0");
19
20        if (grossSales < 0.0) // valida
21            throw new IllegalArgumentException("Gross sales must be >= 0.0");
22
23        this.grossSales = grossSales;
24        this.commissionRate = commissionRate;
25    }
26
27    // configura a quantidade de vendas brutas
28    public void setGrossSales(double grossSales)
29    {
30        if (grossSales < 0.0) // valida
31            throw new IllegalArgumentException("Gross sales must be >= 0.0");
32
33        this.grossSales = grossSales;
34    }
35

```

```

36    // retorna a quantidade de vendas brutas
37    public double getGrossSales()
38    {
39        return grossSales;
40    }
41
42    // configura a taxa de comissão
43    public void setCommissionRate(double commissionRate)
44    {
45        if (commissionRate <= 0.0 || commissionRate >= 1.0) // valida
46            throw new IllegalArgumentException(
47                "Commission rate must be > 0.0 and < 1.0");
48
49        this.commissionRate = commissionRate;
50    }
51
52    // retorna a taxa de comissão
53    public double getCommissionRate()
54    {
55        return commissionRate;
56    }
57
58    // calcula os rendimentos; sobreescreve o método earnings em Employee
59    @Override
60    public double earnings()
61    {
62        return getCommissionRate() * getGrossSales();
63    }
64
65    // retorna a representação String do objeto CommissionEmployee
66    @Override
67    public String toString()
68    {
69        return String.format("%s: %s\n%s: $%.2f; %s: %%.2f",
70            "commission employee", super.toString(),
71            "gross sales", getGrossSales(),
72            "commission rate", getCommissionRate());
73    }
74 } // fim da classe CommissionEmployee

```

```
1 // Figura 10.8: BasePlusCommissionEmployee.java
2 // Classe BasePlusCommissionEmployee estende a CommissionEmployee.
3
4 public class BasePlusCommissionEmployee extends CommissionEmployee
5 {
6     private double baseSalary; // salário de base por semana
7
8     // construtor
9     public BasePlusCommissionEmployee(String firstName, String lastName,
10         String socialSecurityNumber, double grossSales,
11         double commissionRate, double baseSalary)
12     {
13         super(firstName, lastName, socialSecurityNumber,
14             grossSales, commissionRate);
15
16         if (baseSalary < 0.0) // valida baseSalary
17             throw new IllegalArgumentException("Base salary must be >= 0.0");
18
19         this.baseSalary = baseSalary;
20     }
21
22     // configura o salário-base
23     public void setBaseSalary(double baseSalary)
24     {
25         if (baseSalary < 0.0) // valida baseSalary
26             throw new IllegalArgumentException("Base salary must be >= 0.0");
27
28         this.baseSalary = baseSalary;
29     }
30
31     // retorna o salário-base
32     public double getBaseSalary()
33     {
34         return baseSalary;
35     }
36
37     // calcula os vencimentos, sobreescreve o método earnings em CommissionEmployee
38     @Override
39     public double earnings()
40     {
41         return getBaseSalary() + super.earnings();
42     }
43
44     // retorna a representação String do objeto BasePlusCommissionEmployee
45     @Override
46     public String toString()
47     {
48         return String.format("%s %s: %s: %12.2f",
49             "base-salaried", super.toString(),
50             "base salary", getBaseSalary());
51     }
52 } // fim da classe BasePlusCommissionEmployee
```

```
1 // Figura 10.9: PayrollSystemTest.java
2 // Programa de teste da hierarquia Employee.
3
4 public class PayrollSystemTest
5 {
6     public static void main(String[] args)
7     {
8         // cria objetos de subclasse
9         SalariedEmployee salariedEmployee =
10             new SalariedEmployee("John", "Smith", "111-11-1111", 800.00);
11         HourlyEmployee hourlyEmployee =
12             new HourlyEmployee("Karen", "Price", "222-22-2222", 16.75, 40);
13         CommissionEmployee commissionEmployee =
14             new CommissionEmployee(
15                 "Sue", "Jones", "333-33-3333", 10000, .06);
16         BasePlusCommissionEmployee basePlusCommissionEmployee =
17             new BasePlusCommissionEmployee(
18                 "Bob", "Lewis", "444-44-4444", 5000, .04, 300);
19
20         System.out.println("Employees processed individually:");
21
22         System.out.printf("Employee: %s\n",
23             salariedEmployee, "earned", salariedEmployee.earnings());
24         System.out.printf("Employee: %s\n",
25             hourlyEmployee, "earned", hourlyEmployee.earnings());
26         System.out.printf("Employee: %s\n",
27             commissionEmployee, "earned", commissionEmployee.earnings());
28         System.out.printf("Employee: %s\n",
29             basePlusCommissionEmployee,
30             "earned", basePlusCommissionEmployee.earnings());
31
32         // cria um array Employee de quatro elementos
33         Employee[] employees = new Employee[4];
34
35         // inicializa o array com Employees
36         employees[0] = salariedEmployee;
37         employees[1] = hourlyEmployee;
38         employees[2] = commissionEmployee;
39         employees[3] = basePlusCommissionEmployee;
```

Prestem atenção
as linhas 33 a 39:
podemos atribuir
as referências das
subclasses
a variáveis da
superclasse
Employee, *mesmo*
que Employee *seja*
uma classe abstrata.
Polimorfismo!!!



UNIVALI

```
40
41 System.out.printf("Employees processed polymorphically:\n\n");
42
43 // processa genericamente cada elemento no employees
44 for (Employee currentEmployee : employees)
45 {
46     System.out.println(currentEmployee); // invoca toString
47
48     // determina se elemento é um BasePlusCommissionEmployee
49     if (currentEmployee instanceof BasePlusCommissionEmployee)
50     {
51         // downcast da referência de Employee para
52         // referência a BasePlusCommissionEmployee
53         BasePlusCommissionEmployee employee =
54             (BasePlusCommissionEmployee) currentEmployee;
55
56         employee.setBaseSalary(1.10 * employee.getBaseSalary());
57
58         System.out.printf(
59             "new base salary with 10%% increase is: $%,.2f\n",
60             employee.getBaseSalary());
61     } // fim do if
62
63     System.out.printf(
64         "earned $%,.2f\n\n", currentEmployee.earnings());
65 } // for final
66
67 // obtém o nome do tipo de cada objeto no array employees
68 for (int j = 0; j < employees.length; j++)
69     System.out.printf("Employee %d is a %s\n", j,
70         employees[j].getClass().getName());
71 } // fim de main
72 } // fim da classe PayrollSystemTest
```

Linhas 44 a 65 iteram pelo array e invocam os métodos `toString` e `earnings` de um diferente `Employee` no array a cada iteração.

Linha 49 um *cast* é feito (cláusula `instanceof`).

Linha 70 o método `getClass` retorna objeto do tipo `Class` (do pacote `java.lang`), contendo informações sobre o tipo do objeto, incluindo seu nome de classe.

Employees processed individually:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: \$800.00
earned: \$800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: \$16.75; hours worked: 40.00
earned: \$670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: \$10,000.00; commission rate: 0.06
earned: \$600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: \$5,000.00; commission rate: 0.04; base salary: \$300.00
earned: \$300.00

Employees processed polymorphically:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: \$800.00
earned \$800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: \$16.75; hours worked: 40.00
earned \$670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: \$10,000.00; commission rate: 0.06
earned \$600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: \$5,000.00; commission rate: 0.04; base salary: \$300.00
new base salary with 10% increase is: \$330.00
earned \$330.00

Employee 0 is a SalariedEmployee
Employee 1 is a HourlyEmployee
Employee 2 is a CommissionEmployee
Employee 3 is a BasePlusCommissionEmployee

A saída ilustra que os métodos específicos para cada classe foram de fato invocados. Todas as chamadas ao método `toString` e `earnings` são resolvidas em tempo de execução, com base no tipo do objeto que `currentEmployee` referencia.

Ligação tardia ou ligação dinâmica!!
(*late/dynamic biding*)

Java

Polimorfismo



- Pode-se usar *casting* para forçar o compilador a entender que um objeto da superclasse na verdade é um objeto de uma subclasse...

```
((Suclasse) (arraySuper.get(i))).metodoSub()
```

- Mas o Java dará um erro em tempo de execução se o objeto não for uma instância da referida subclasse...
- A forma correta para fazer isso é utilizando a cláusula `instanceof` para se assegurar de que o *cast* é possível em tempo de execução.
- Entretanto, perde-se o polimorfismo e suas vantagens.

Java

Polimorfismo



- Um objeto de **subclasse** é também um objeto de **superclasse**, mas as duas classes são diferentes!!
- Como a **subclasse** pode ter membros adicionais somente dela, atribuir uma referência de **superclasse** a uma variável de **subclasse** não é permitido sem uma *coerção explícita*.
- Atribuir uma referência de **subclasse** a uma variável de **superclasse** é seguro, porque o objeto de **subclasse** **é um** objeto de sua **superclasse**.
 - Mas, a variável da **superclasse** pode ser utilizada para referenciar *apenas* membros da **superclasse**.
 - Se o código referenciar membros somente da **subclasse** por meio da variável de **superclasse**, o compilador dará erro.

Java

Polimorfismo



Chamada de métodos a partir de construtores:

- Não chame métodos que podem ser sobrescritos a partir de construtores.
 - o objeto da subclasse pode não estar totalmente inicializado e isso pode levar a erros sutis difíceis de detectar caso o método da subclasse chamado depender desta inicialização (que ainda não foi realizada) no construtor da subclasse.

Java

Polimorfismo



Chamada de métodos a partir de construtores:

- É aceitável chamar um método `static` a partir de um construtor.
 - o construtor e o método `set` muitas vezes fazem a mesma validação para um objeto específico. Se o código de validação for curto, é aceitável duplicá-lo no construtor e no método `set`, mas se for longo, defina um método de validação `static` (normalmente um método auxiliar `private`) e, então, chame-o no construtor e no método `set`.
- Também é aceitável que um construtor chame um método de instância `final`, desde que este método não chame direta ou indiretamente um outro método de instância que pode ser sobrescrito.

Ligação (*Binding*)

- Alguns autores utilizam o termo “acoplamento” ou vinculação como tradução de ***binding***.
- Refere-se a associação entre classe e método.
- Um método associado com uma classe é dito estar *ligado* (*bound*) à classe.
- Métodos podem ser ligados a uma classe em tempo de compilação (*early binding*) ou a um objeto em tempo de execução (*late binding*).

Ligação (*Binding*)



- **Ligação prematura (*early binding*) ou ligação estática (*static binding*):**
 - Realizada em tempo de compilação.
 - Ocorre quando o método a ser invocado é definido durante a compilação do programa.
 - Problema: programador precisa saber quais objetos serão usados para as chamadas dos seus métodos.
 - Utilizada em linguagens convencionais/tradicionais.
 - Vantagem: eficiência computacional.

Ligação (*Binding*)



- **Ligação tardia (*late binding*) ou ligação dinâmica (*dynamic binding*):**
 - Realizada em tempo de execução.
 - Ocorre quando o método a ser invocado é definido somente em tempo de execução do programa.
 - Mecanismo necessário para polimorfismo.
 - Vantagem: flexibilidade.
 - Ligação tardia em Java:
 - comportamento padrão
 - métodos `final` não podem ser redefinidos e o polimorfismo não se aplica
 - métodos `private` são implicitamente declarados como `final`

Orientação a Objetos

Tipos de Polimorfismo



- **UNIVERSAL** (verdadeiro): quando uma operação ou tipo **trabalha uniformemente** para uma gama de tipos definidos na linguagem.
 - Inclusão (tipo mais básico que existe): um ponteiro para classe mãe pode apontar para uma instância de uma classe filha.
 - Paramétrico: se restringe ao uso de *templates* (C++) e *generics* (C#/Java).
- **AD-HOC** (aparente): quando uma operação ou tipo **parece trabalhar** para alguns tipos diferentes e pode se comportar de formas diferentes para cada tipo.
 - Sobrecarga: duas funções/métodos com o mesmo nome mas assinaturas diferentes.
 - Coerção: conversão implícita de tipos sobre os parâmetros de uma função.

Exercícios

- Resolver os exercícios abaixo (fonte Deitel,2017).

- 10.12** (*Modificação do sistema de folha de pagamento*) Modifique o sistema de folha de pagamento das figuras 10.4 a 10.9 para incluir a variável de instância `private birthDate` na classe `Employee`. Utilize a classe `Date` da Figura 8.7 para representar o aniversário de um funcionário. Adicione métodos *get* à classe `Date`. Suponha que a folha de pagamento seja processada uma vez por mês. Crie um array de variáveis `Employee` para armazenar referências aos vários objetos de funcionário. Em um loop, calcule a folha de pagamento para cada `Employee` (polimorficamente) e adicione um bônus de US\$ 100.00 à quantia da folha de pagamento do funcionário se o mês atual for aquele em que ocorre o aniversário do `Employee`.
- 10.14** (*Modificação do sistema de folha de pagamento*) Modifique o sistema de folha de pagamento das figuras 10.4 a 10.9 para incluir uma subclasse `PieceWorker` adicional de `Employee` que representa um funcionário cujo pagamento está baseado no número de peças de mercadorias produzido. A classe `PieceWorker` deve conter variáveis de instância `wage private` (para armazenar o salário do funcionário por peça) e `pieces` (para armazenar o número de peças produzido). Forneça uma implementação concreta do método `earnings` na classe `PieceWorker` que calcula os vencimentos do funcionário multiplicando o número de peças produzido pelo salário por peças. Crie um array de variáveis `Employee` para armazenar referências a objetos de cada classe concreta na nova hierarquia `Employee`. Para cada `Employee`, exiba sua representação de `String` e vencimentos.